



Universidade Estadual do Piauí - UESPI
Campus Alexandre Alves de Oliveira - Parnaíba/PI
Bacharelado em Ciência da Computação



Modelo Ágil

Átila Rabelo Lopes
Engenharia de Software

Modelo Ágil

- Métodos tradicionais de desenvolvimento de software
 - Rígidos e burocráticos
 - Documentação extensa
 - Planejamento minucioso e detalhado (desde o início do projeto)
 - Resultava em atrasos e softwares abaixo das expectativas dos clientes
- Manifesto Ágil foi uma alternativa aos métodos tradicionais

“O software funcionando e agregando valor ao cliente é mais importante que a documentação extensa”

Modelo Ágil



Errado – Metodologia Ágil não é o caos!

Modelo Ágil

Manifesto Ágil

- Documento que revolucionou o desenvolvimento de software
- Elaborado e assinado por 16 renomeados programadores (2001)
- Apresenta um conjunto de valores e princípios para flexibilizar o processo de desenvolvimento de software

Modelo Ágil

Valores do Manifesto Ágil

- **Indivíduos e interações** são mais importantes que processos e ferramentas.
- **Software funcionando** é mais importante do que documentação completa e detalhada
- **Colaboração com o cliente** é mais importante do que negociação de contratos
- **Adaptação a mudanças** é mais importante do que seguir o plano inicial.

Modelo Ágil

Princípios do manifesto ágil

1 - A prioridade é satisfazer o cliente

- Entrega adiantada e contínua de software de valor

2 - Aceitar mudanças de requisitos

- Processos ágeis se adequam as mudanças

3 - Entregar software funcionando com frequência

- Escala de semanas até meses, preferencialmente em períodos mais curtos.

Modelo Ágil

- 4 - Pessoas de negócios e Desenvolvedores devem trabalhar em conjunto e diariamente
- 5 - Construir projetos ao redor de indivíduos motivados
- 6 - O método mais eficiente de transmitir informações é a comunicação pessoal
- 7 - Software funcional é a medida primária de progresso
- 8 - Contínua atenção à excelência técnica e bom design, aumenta a agilidade
- 9 – A equipe deve refletir sobre seu desempenho e se ajustar para melhorar a efetividade, freqüentemente

Modelo Ágil

Equipe Ágil

- Auto-organização
- Controla seu próprio destino
- Enfatiza a comunicação e a colaboração entre todos
- Competente
- Foco comum
- Capacidade de tomada de decisão
- Habilidade de resolver problemas vagos
- Respeito e confiança mútua

Modelo Ágil

Modelo Ágil

- Modelo iterativo e incremental com foco na entrega rápida e incremental de um produto funcional
- Características:
 - Iterações curtas
 - Flexibilidade à mudanças
 - Colaboração do usuário
 - Trabalho em equipe
 - Feedback contínuo

Modelo Ágil

- Iterações curtas:
 - O desenvolvimento é dividido em ciclos curtos "sprints" (1 a 4 semanas)
- Entrega contínua de valor:
 - Entregar uma versão funcional ao final de cada sprint
- Colaboração constante com o cliente:
 - O cliente participa de reuniões, revisões e ajustes nos requisitos durante todo o processo

Modelo Ágil

- Adaptação à mudanças:
 - Flexível à mudanças nos requisitos mesmo em fases avançadas do desenvolvimento
- Equipes multidisciplinares:
 - As equipes são auto-gerenciáveis e multidisciplinares (desenvolvedores, analistas, designers, avaliadores,..)
- Feedback contínuo:
 - Promove feedback constante entre as partes envolvidas (clientes, stakeholders, equipe)

Modelo Ágil

Vantagens

- Flexibilidade e adaptação a mudanças
- Entrega rápida e contínua de funcionalidades
- Foco no cliente e colaboração constante
- Redução de riscos (feedback frequente)
- Autonomia e motivação das equipes

Modelo Ágil

Desvantagens

- Dificuldade em grandes projetos ou equipes distribuídas
- Menor foco em documentação formal
- Necessidade de envolvimento constante do cliente
- Indefinição de escopo e previsão de custos no início
- Dependência de equipes altamente qualificadas

Metodologias Ágeis

- XP (eXtreme Programming)
- SCRUM
- Crystal/Clear
- FDD (*Feature Driven Development*)
 - Desenvolvimento Orientado a Funcionalidades
- DSDM (*Dynamic Systems Development Method*)
 - Método Dinâmico de Desenvolvimento de Sistemas
- ASD (*Adaptive Software Development*)
 - Desenvolvimento de Software Adaptável

SCRUM

Scrum

- Uma das metodologias ágeis mais populares
- Usada para o gerenciamento de projetos / desenvolvimento de software
- Adota uma abordagem iterativa e incremental
 - Entrega rápida: Sprints curtos (1 a 4 semanas)
 - Entrega contínua: Incremento a cada sprint

Scrum

Papéis

- **Product Owner (Dono do Produto):**
 - Responsável por maximizar o valor do produto. Define as funcionalidades e prioriza as tarefas no “Product Backlog”
- **Scrum Master:**
 - Atua como facilitador do processo Scrum
 - Garante que a equipe siga os princípios ágeis e gerencia todo o processo de desenvolvimento
- **Time de Desenvolvimento:**
 - Equipe multifuncional e auto-organizada
 - Responsável por entregar incrementos de produto ao final de cada Sprint
 - Geralmente tem de 3 a 9 membros.

Scrum

Eventos do Scrum

- **Planejamento da Sprint (Sprint Planning):**
 - Reunião no início de cada Sprint para definir as tarefas (Product Backlog) e como executá-las
- **Scrum Diário (Daily Scrum):**
 - Reunião diária de 15 minutos (sincronizar as atividades e ajustar o plano de trabalho posterior)
- **Revisão da Sprint (Sprint Review):**
 - Reunião no final do Sprint (mostrar o incremento, obter feedback e adaptar o “Product Backlog”)
- **Retrospectiva da Sprint (Sprint Retrospective):**
 - Reunião para refletir sobre o que funcionou bem e o que precisa ser melhorado no próximo Sprint

Scrum

Artefatos

- **Product Backlog:**
 - Lista priorizada de tudo que é necessário para o produto
- **Sprint Backlog:**
 - Lista de tarefas selecionadas do Product Backlog (que serão executadas durante a Sprint)
- **Incremento:**
 - Conjunto de todos os itens do *Product Backlog* concluídos na Sprint (pronto para uso)

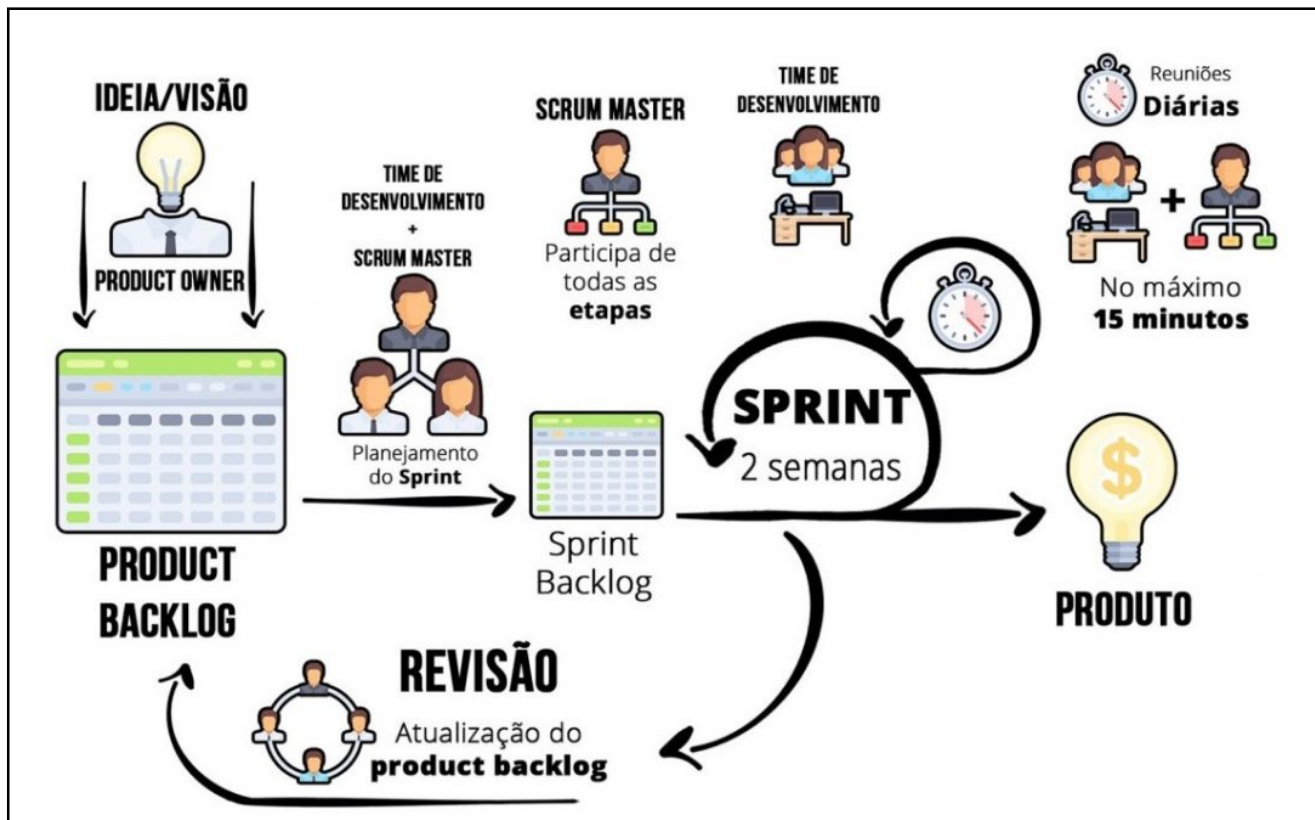
Scrum

Princípios

- **Transparência:**
 - Todos os aspectos do processo estão visíveis para quem for necessário
- **Inspeção:**
 - Inspeções frequentes para identificar problemas ou desvios
- **Adaptação:**
 - Ajustes são feitos para garantir que o projeto continue no caminho certo

Valores

- Compromisso, coragem, foco, abertura e respeito



Scrum

ETAPAS SCRUM

- Criação e Gestão do Product Backlog
 - Product Backlog:
 - Lista priorizada dos requisitos, funções e correções do produto (histórias de usuário)
 - Gerenciado pelo Product Owner
- Planejamento da Sprint (Sprint Planning)
 - Define o objetivo e o trabalho que será feito durante a Sprint
 - O Product Owner apresenta os itens prioritários (Product Backlog)
 - A equipe de desenvolvimento:
 - Seleciona os itens do Product Backlog que serão entregues na Sprint
 - Define um plano detalhado para atingir o objetivo (Sprint)

Scrum

- **Execução do Sprint**
 - Execução dos itens selecionados (Sprint Backlog), seguindo o plano de trabalho definido
 - Duração do Sprint é fixa e geralmente varia de 1 a 4 semanas, dependendo do projeto.
- **Scrum Diário (Daily Scrum)**
 - Reunião diária de 15 minutos
 - Cada membro responde: o que fez ontem, o que vai fazer hoje e se há algum impedimento
 - O objetivo é identificar e resolver problemas rapidamente, garantindo que a equipe esteja alinhada.
- **Revisão da Sprint (Sprint Review)**
 - Ocorre no final do Sprint e tem como objetivo inspecionar o incremento do produto
 - O time apresenta o trabalho concluído para os stakeholders, que fornecem feedback
 - Com base no feedback, o Product Owner pode atualizar o Product Backlog

Scrum

- **Retrospectiva da Sprint (Sprint Retrospective)**
 - Visa melhorar o processo de trabalho
 - A equipe reflete sobre a execução das tarefas da sprint
 - Toma as decisões necessárias para melhorar na próxima Sprint
- **Reinício do Ciclo**
 - O ciclo Scrum é repetido com a próxima Sprint
 - O Product Backlog é atualizado (novos requisitos e prioridades)
 - As etapas são repetidas para cada Sprint

XP (eXtreme Programming)

XP (eXtreme Programming)

- Evolução do modelo incremental
- Amplamente conhecida e utilizada em projetos de software
- Voltada para equipes de até 20 pessoas
- Adequada para projetos de software cujos requisitos são vagos e/ou mudam constantemente
- Objetivos:
 - Melhorar a qualidade do software
 - Atender as mudanças de requisitos dos clientes
 - Promover o desenvolvimento rápido, incremental e adaptável

XP (eXtreme Programming)

Características

- Desenvolvimento/entrega de incrementos de pequenas funcionalidades
- Envolvimento do cliente no processo
- Constante melhoria de código
- Programação em pares
- 40 horas de trabalho.
- *Refactoring*
 - Abordagem disciplinada para tornar o código de um software mais claro e de fácil manutenção, minimizando a probabilidade de inclusão de erros.
- *Test first, code later.*

XP (eXtreme Programming)

- Adota um conjunto de valores e práticas seguidos rigorosamente
- Valores
 - Comunicação
 - Simplicidade
 - Feedback
 - Coragem
 - Respeito

XP (eXtreme Programming)

Valores

- **Comunicação:**
 - A equipe deve se comunicar de forma contínua (reuniões frequentes)
- **Simplicidade:**
 - A equipe desenvolve apenas o necessário para as necessidades atuais, sem adicionar funcionalidades desnecessárias
- **Feedback:**
 - O feedback rápido e contínuo vem dos testes automatizados, do cliente e da própria equipe
- **Coragem:**
 - A equipe deve estar disposta a tomar decisões ousadas, como refatorar o código para melhorar a estrutura e eliminar dívidas técnicas, sem medo de errar ou mudar o curso.
- **Respeito:**
 - Todos da equipe são respeitados e suas idéias e contribuições são valorizadas

XP (eXtreme Programming)

Práticas

- **Programação em Par (Pair Programming)**
 - Dois programadores trabalham juntos em uma única estação de trabalho.
 - Um escreve o código enquanto o outro revê e sugere melhorias.
- **Desenvolvimento Guiado por Testes (TDD)**
 - Antes de implementar qualquer funcionalidade, os programadores escrevem testes automatizados que definem o comportamento esperado do software.
 - Somente depois o código é escrito para passar nesses testes
- **Refatoração Contínua**
 - Refatoração é o processo de melhorar a estrutura interna do código, sem alterar seu comportamento externo.

XP (eXtreme Programming)

- Integração Contínua
 - O código deve ser integrado e testado várias vezes ao dia
 - Alterações feitas por diferentes desenvolvedores são constantemente unidas em um repositório central (ajuda a identificar conflitos e erros o mais cedo possível)
- Lançamentos Pequenos e Frequentes
 - Entregas frequentes de software funcional, em ciclos curtos (uma semana a um mês)
- Metáfora
 - Em vez da documentação detalhada, o XP usa metáforas para descrever como o sistema deve funcionar
 - As metáforas são usadas pela equipe para melhorar sua compreensão sobre o funcionamento do software e as interações internas entre os componentes.

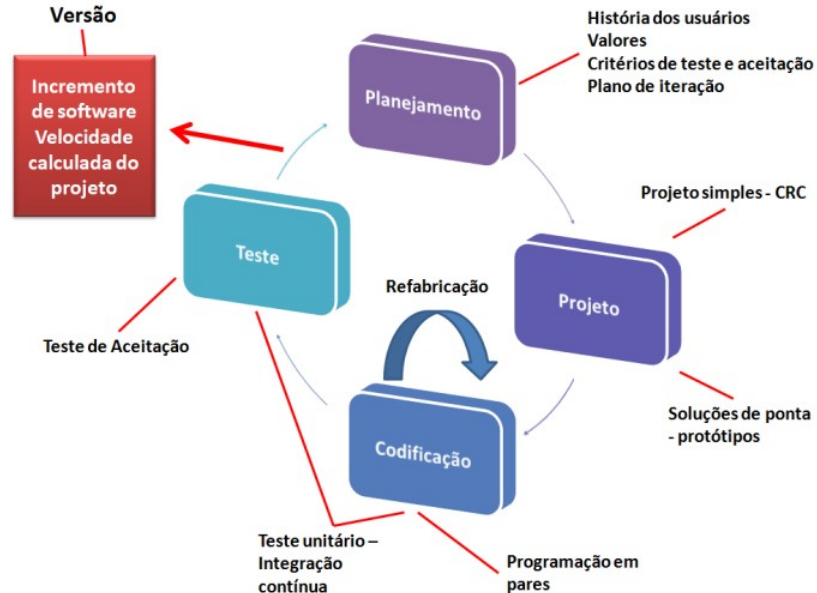
XP (eXtreme Programming)

- Propriedade Coletiva do Código
 - O código é de propriedade de toda a equipe
 - Qualquer membro da equipe pode alterar qualquer parte do código
- Cliente Sempre Presente
 - O cliente (ou representante) deve estar sempre disponível à equipe para fornecer feedback imediato e ajudar na definição de requisitos
- Padrões de Codificação
 - A equipe segue um conjunto comum de padrões de codificação para garantir consistência no estilo e na estrutura do código

XP (eXtreme Programming)

Etapas:

- Planejamento
- Projeto
- Codificação
- Teste



XP (eXtreme Programming)

Planejamento (Planning)

- Início do processo onde a equipe e o cliente colaboram para definir as funcionalidades e estabelecer as prioridades para as próximas iterações
- Principais atividades:
 - **Histórias do Usuário:**
 - O cliente escreve pequenas descrições das funcionalidades desejadas, chamadas de histórias de usuário, que expressam as necessidades e expectativas do software.
 - **Priorização:**
 - O cliente prioriza as histórias de usuário com base no valor para o negócio e na urgência. A equipe de desenvolvimento avalia a complexidade técnica de cada história.
 - **Criação do Release Plan (Plano de Lançamento):**
 - A equipe cria um plano de entrega das funcionalidades para as próximas iterações (sprints). O plano é flexível e pode mudar com base no feedback.

XP (eXtreme Programming)

Projeto (Design)

- Construção de um design simples e direto para atender às necessidades imediatas das histórias do usuário
 - Design Simples:
 - A equipe cria o design mais simples possível para implementar as funcionalidades prioritárias, sem complicações desnecessárias.
 - Metáforas:
 - Em vez de documentação extensa, a equipe utiliza metáforas para explicar como o sistema funciona, facilitando o entendimento por todos os membros.
 - Refatoração:
 - A refatoração contínua é usada para melhorar o design do código ao longo do tempo, garantindo que ele continue simples e eficiente.

XP (eXtreme Programming)

Codificação (Coding)

- Programação em Par (Pair Programming)
 - Dois programadores trabalham juntos no mesmo código, aumentando a qualidade e a troca de conhecimento.
- Propriedade Coletiva do Código
 - Todos os desenvolvedores podem modificar qualquer parte do código, promovendo a colaboração e a responsabilidade compartilhada.
- Integração Contínua
 - O código é integrado e testado várias vezes ao dia para garantir que todas as mudanças se ajustem ao sistema e que bugs sejam encontrados rapidamente.

XP (eXtreme Programming)

Testes (Testing)

- Os testes são uma parte central do XP
- A qualidade do software é assegurada com robustos testes automatizados
- Realizados em diferentes níveis:
 - Test-Drive Development (TDD)
 - Testes de Aceitação
 - Testes Automatizados

XP (eXtreme Programming)

- Test-Driven Development (TDD)
 - Os programadores escrevem os testes antes de escrever o código
 - O código é desenvolvido para passar nos testes (atende aos requisitos)
- Testes de Aceitação
 - O cliente define testes de aceitação para garantir que as histórias de usuário implementadas atendam às expectativas
- Testes Automatizados
 - Automatiza os testes unitários (partes pequenas do código) e testes de aceitação

XP (eXtreme Programming)

Lançamento (Release)

- A cada iteração é entregue um incremento para o cliente usar e testar
- Ciclos Curtos de Desenvolvimento (geralmente entre 1 a 3 semanas)
- O cliente acompanha o progresso e fornece feedback constante
- Entrega constante de novas funcionalidades do software (cada iteração)

XP (eXtreme Programming)

Feedback e Revisão (Review and Feedback)

- Feedback contínuo do cliente sobre as funcionalidades entregues (incrementos)
- O feedback é usado para ajustar o planejamento da próxima iteração
- A equipe revisa o código e os processos internos em busca de melhorias

Refatoração Contínua (Continuous Refactoring)

- O código é revisado e melhorado continuamente
 - O código é reescrito para ser mais claro, limpo e de fácil manutenção, sem alterar seu comportamento
 - Remove códigos duplicados e complexidades desnecessárias

XP (eXtreme Programming)

Histórias do Usuário

- Uma das principais formas de capturar os requisitos de software no XP
- Descrições curtas e simples (linguagem natural) das funcionalidades/recursos que o usuário deseja no produto final
 - Necessidades e expectativas do usuário
- Ajudam a comunicação entre desenvolvedores e stakeholders
- Importantes para priorizar e planejar as entregas em ciclos curtos

Exemplos

1. Sistema de E-commerce

- História do usuário:
 - "Como cliente, quero adicionar produtos ao carrinho de compras para que eu possa revisá-los e comprá-los depois."
- Contexto:
 - Essa história reflete o desejo do cliente de poder adicionar itens ao carrinho enquanto navega pelo site.
- A funcionalidade deve permitir:
 - Adicionar, remover ou modificar a quantidade de produtos no carrinho.
- Critérios de aceitação:
 - O cliente pode adicionar produtos ao carrinho de qualquer página do site.
 - O carrinho deve exibir o total de itens e o valor parcial.
 - O cliente pode revisar o carrinho antes de finalizar a compra.

Exemplos

2. Aplicação Bancária

- História do usuário:
 - "Como usuário, quero visualizar meu saldo e extrato bancário em tempo real."
- Contexto:
 - O usuário deseja acompanhar suas finanças com precisão e rapidez
- A funcionalidade deve permitir:
 - Exibir o saldo atualizado e um histórico de transações
- Critérios de aceitação:
 - O saldo deve ser atualizado em tempo real.
 - O extrato deve exibir as transações dos últimos 30 dias, com data e descrição.
 - O usuário deve poder filtrar transações por tipo (depósitos, saques, transferências).

Exemplos

3. Sistema de suporte ao cliente

- História do usuário:
 - "Como cliente, quero abrir um chamado de suporte para relatar problemas técnicos e acompanhar o status da resolução."
- Contexto:
 - O cliente quer um canal eficiente para relatar problemas e monitorar o andamento da solução.
- A funcionalidade deve permitir:
 - - Abrir chamados e fornecer atualizações
- Critérios de aceitação:
 - O cliente pode criar um chamado com detalhes do problema.
 - O cliente recebe uma notificação quando o status do chamado mudar (em análise, resolvido, etc.).
 - O sistema exibe um histórico de chamados abertos.

XP (eXtreme Programming)

Metáforas

- Mecanismo para facilitar a comunicação e a compreensão entre os membros da equipe de desenvolvimento e os stakeholders
- Descrição simplificada de como o sistema funciona, utilizando comparações que são familiares para todos os envolvidos
- Permite que todos os envolvidos (equipe/cliente) possam entender o projeto como um todo

Exemplo

Metáfora: Sistema de E-commerce

- Metáfora: "Carrinho de Supermercado"
- O sistema de e-commerce pode ser comparado a um supermercado físico
 - Adiciona os produtos que deseja comprar
 - Ao terminar, vai para o caixa do supermercado
 - Revisa os itens no carrinho
 - Faz o pagamento
 - Retira os produtos